Don't Interrupt the Thinker

Reasoning Efficiency, Imitation, and Memory in ARC-AGI-3

Xinyi Fang

Abstract

Give a language-model agent less time to think and it acts more, but on ARC-AGI-3 it does not win more. We made six changes to how an agent's reasoning is spent, shortened, interrupted, or remembered, and played each on the 25 public games. A checkpoint trained to answer briefly led its base model at three hours, then finished ten hours 15 levels behind despite taking twice as many actions. A prompt that stopped the agent from re-analyzing an unchanged board halved its stalled time and added 46% more actions, but only two more levels and 2.3 times as many game overs. Restarting the agent's context earlier in long attempts cost about nine levels, although two in five attempts that had already run past 40,000 tokens still went on to clear the level. Fine-tuning on winning play made attempts shrink from 82 actions to 25, and first-level success on held-out games fell from 47% to 11%. In the other direction, letting the server deliver more reasoning lifted completions from 41 to 73 levels across four paired runs, and restoring reasoning that a chat template had silently stripped tripled the score and reversed which sampling temperature looked best. Extra turns helped only when they did not come out of the thinking. For an agent that must work out unfamiliar rules, the expensive part of a turn is the useful part.

1 Introduction

A player sees a colored grid and a set of available controls. Pressing a key or clicking a cell may move an object, change a color, or leave the board looking much as it did before. The game supplies no instructions. To finish a level, the player has to work out both what an action does and which change is worth pursuing. ARC-AGI-3 turns that problem into a benchmark for acquiring skills through interaction [1, 2], after two years of ARC Prize competitions on static puzzles [3].

A language-model player has to decide how to spend its time: reason longer about what it has seen, or act now and learn from the result. Under a fixed time budget, a model that thinks less per turn can take more turns, and more turns look like more exploration. We tested that trade six ways, each a change that shortens, interrupts, extends, or preserves the agent's reasoning, and the results line up (Table 1). Changes that cut the thinking short lost levels or gained almost none. The two changes that let the model think longer, or remember what it had already thought, gained.

The sharpest case is Swift [4], a version of Qwen3.8-27B [5] trained to generate shorter responses. In the same game-playing software it took more than twice as many actions as its parent and led after three hours. Seven hours later it was 15 levels behind (Figure 1).

The other changes tell the same story from different sides. When a notice pushed a stalled agent to stop re-analyzing and act, its stalled time fell from about half of play to a fifth and it took 46% more actions, yet it cleared only two more levels and ran into 2.3 times as many game overs. Restarting the agent's context earlier in long attempts cost about nine levels, because two in five attempts that had already run past 40,000 tokens were still on their way to clearing the level. Fine-tuning on winning play taught the agent to play short, and its first-level success on new games fell from 47% to 11%. In the other direction, a less congested server let the same model finish more complete decisions and lifted completions from 41 to 73 levels, and restoring the reasoning that a chat template had been deleting tripled the score.

Change	What it did to the thinking	Result	Sec.
Less thinking			
Swift checkpoint	47% fewer tokens per turn	14.9 fewer levels at 10 h	3
Retry notice	stalled time cut from about half to a fifth	2 more levels, 2.3× the game overs	5
Earlier restart	long attempts wiped at 49,152 tokens	8.7 fewer levels	7
Training on wins	attempts cut from 82 to 25 actions	first-level success 47% → 11%	8
More thinking			
More serving capacity	3.6× the tokens, similar tokens per action (one pair)	41 → 73 levels over four pairs	4
Keeping earlier reasoning	the model sees its past analysis	3.2× the game score at temperature 0.6	6

Table 1. Changes that cut the thinking short lost levels or gained almost none; the two that gave the model more of its own reasoning gained. Each row is a separate experiment on the 25 public games.

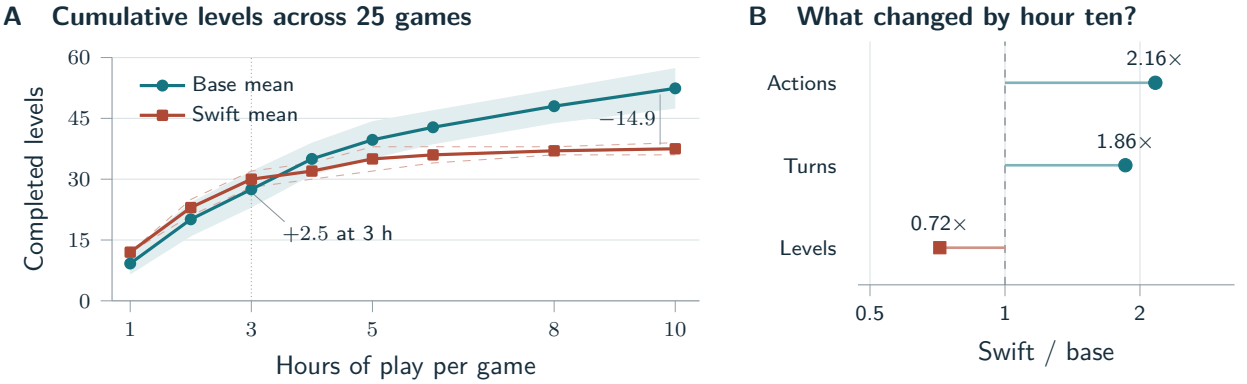


Figure 1. The three-hour leader clears fewer levels by ten hours. In A, the pale dashed lines are the two individual Swift runs. The base curve joins means from earlier runs ($n = 23$ at ten hours), and the shaded band spans one standard deviation across those runs at each hour. Labels show Swift minus base in completed levels. In B, each point compares the two-run Swift mean with the corresponding base mean. A turn counts here when the model produces at least one game action. One turn can produce several actions.

2 What the player sees and what a move costs

An ARC-AGI-3 environment presents a 64×64 grid drawn from 16 colors. Each game enables a subset of the controls, which can include a coordinate-based click. The player receives the resulting frame or sequence of frames. Clearing a level advances the environment and provides direct evidence that a particular course of action met its goal [1]. Until then, the consequences of a move are often easier to observe than its strategic value. Tracking the same cluster of pixels before and after a keypress, for instance, may reveal which object the key controls. It still leaves the desired destination open.

The mechanics are the easy part. The games are deterministic: replaying the same 300 actions from a reset reproduced every frame. In five of six games we examined, a few hundred random moves were enough to pin down what happens to every changed cell from the cells around it and the action, given a window 7 to 41 cells across (Appendix B.1). What those moves do not reveal is the goal, and that is where an agent's reasoning goes.

Our agents combine a language model with software that turns its output into game moves and prepares the next observation. We use the Duck solver and Daniel Franzen's extension of it [6, 7]. Our leaderboard entry (submission 56836202, public score 33.09) runs Franzen's solver with

Qwen3.8-Flash-Next, and the checkpoint and restart experiments in Sections 3 and 7 run this same solver with Qwen3.8-27B in place of Flash-Next (Appendix A.1). The solver supplies the current observation and interaction history to a language model and executes the Python code it returns. Code can inspect a board, compare two observations, or issue an action to the environment. For example, a short Python expression can count the pixels that changed across two existing frames without spending a game action. A call that clicks a cell advances the game, and a loop can issue several actions from one model response.

The difference between inspecting and acting matters when an agent appears to be stuck. It can use several requests to analyze the same position while the environment waits. The game charges actions only when the solver sends a move, but wall time also passes during analysis and while a request waits for inference. Because the 25 games share one inference server, the solver gives each decision step a fixed *turn allowance*; a step that has not issued a move by then yields, and the game retries later. We distinguish a *game action* from an *action-producing turn*, a decision step that executes at least one move. The reported token counts include generated output accumulated within such a turn.

Several different objects can survive until the next turn. The model's previous reasoning is text from its earlier response. A *rule note* is a short description of the game carried into a later prompt. In the Franzen solver, Python functions can also persist and be called again. The game board, the action history, and a record of previous deaths provide additional state. An intervention that removes one of these objects may leave the others available.

Each run plays all 25 public games. We report the total number of levels completed, or the benchmark's game score, which runs from 0 to 100 per game and also weighs how many actions a player spends relative to human play [1]; we compute it locally and report its mean over the 25 games. Appendix A gives hardware, budgets, and sampling for each experiment.

3 The race changes after hour three

3.1 Changing the checkpoint

Swift-1.5-Qwen3.8-27B is Qwen3.8-27B post-trained to cut excessive generation while keeping its performance on reasoning, coding, and agentic tasks [4]. We ran Swift twice in the same Franzen solver as the base model, for ten hours per game, and compared it with 23 earlier base-model runs on the same hardware and solver.

Swift did think less and act more. Its median output per action-producing turn was 1,464 and 1,237 tokens in the two runs, against 2,563 for the base model. It completed 1,649 and 1,673 such turns, against 895, and took 9,075 and 8,572 game actions, against 4,077.

3.2 Early levels sooner, deep levels never

At three hours, the Swift runs had cleared 32 and 28 levels. The base mean was 27.5. By four hours, the same runs had cleared 34 and 30, while the base mean was 35. At ten hours they finished at 39 and 36, against 52.4 for the base: 28.4% fewer completed levels.

By hour ten Swift trailed in 20 of the 25 games. The largest gaps came in games where the base model reached deep levels, such as `sb26`, `tr87`, and `cd82`. Shorter responses bought the early levels sooner; they did not buy the later ones.

The retry experiment in Section 5 keeps the weights fixed, changes only how the agent is prompted, and finds the same pattern.

4 More room to think

If thinking less loses, does thinking more win? Our serving setup offered a direct test. An earlier configuration of our vLLM server [8] disabled prefix caching and admitted at most three requests

at once; in one snapshot of the server, 3 requests were running and 22 were waiting. We enabled caching of previously processed input prefixes and raised the admission limit to 28, and the queue emptied: 25 running, none waiting. Across four pairs the agent cleared 73 levels instead of 41, and three of the four pairs moved in that direction (Figure 2A).

The first pair shows where the gain came from. The agent generated 3.6 times as many tokens and took 3.2 times as many actions, so its reasoning per action barely changed (Figure 2B). It made more complete decisions, not shorter ones. Swift also bought more turns, but by thinking less in each; here the extra turns came from waiting less.

The two halves of the change work together. With eight games and no queue, caching supplied 40–58% of each prompt and added 3.3 levels per pair across nine pairs. When 25 games shared a smaller memory pool and 16–18 requests waited at a time, each game's cached prompt was evicted by the others before its next turn came; once the contexts grew long, the cache supplied almost nothing (Appendix A). A cache only pays when nobody waits.

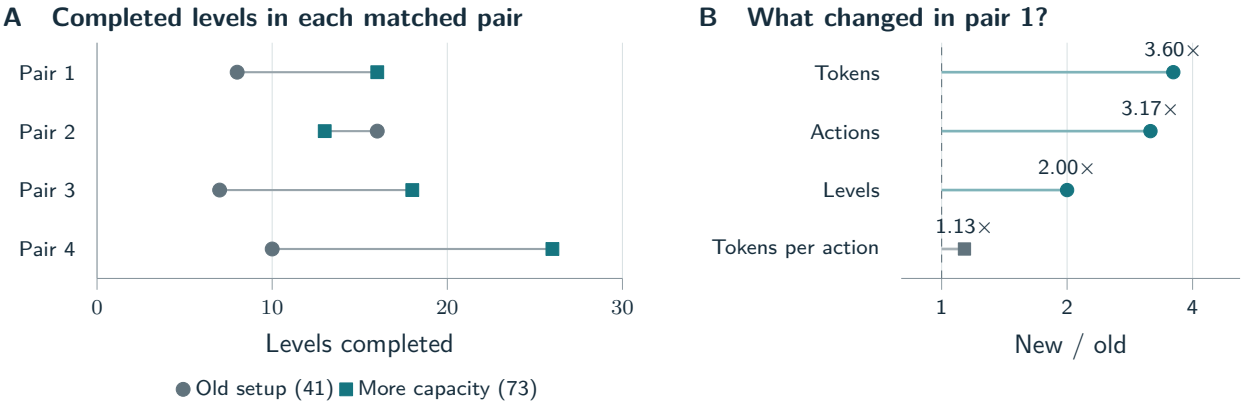


Figure 2. More serving capacity raised completions without shortening the reasoning behind each move. In A, each line joins a matched pair under the old serving setup (no prefix caching, at most three requests at once) and the new one (prefix caching, up to 28 requests). In B, each point divides the new setup's count by the old one's in pair 1, the L40 pair with full logs. Compare Figure 1B, where Swift's extra actions came with fewer levels.

5 Sixteen attempts on the same board

In one diagnostic play session, the solver retried the same step sixteen times over 99 minutes. The record included a repeated 30,220-character reasoning span and no tool call in the repeated response.

When a decision step yielded without an action, the solver rebuilt its user prompt from the board, action counter, and rule note. If none had changed, the user message was identical on the next request. The previous assistant messages could still be present, and we confirmed that the earlier reasoning reached the prompt in this configuration. Thus a solver could see its last line of inquiry, receive the same request again, and resume the same analysis.

5.1 Giving a retry a different starting point

We added a notice to any retry that followed an action-free attempt on an unchanged prompt. It reported the retry count and a short summary of the probes (Python inspections of the board) already tried, and invited the model to choose another probe or a small informative move. The model, game interface, and sampling stayed the same (Appendix A).

We ran the notice and ordinary prompting side by side on the same GPUs, in four pairs on all 25 games with 132 minutes per game. We define *retry occupancy* as the share of wall time in steps

requiring at least three consecutive attempts on an unchanged user prompt. This broad definition includes time that an agent may have spent on useful thinking.

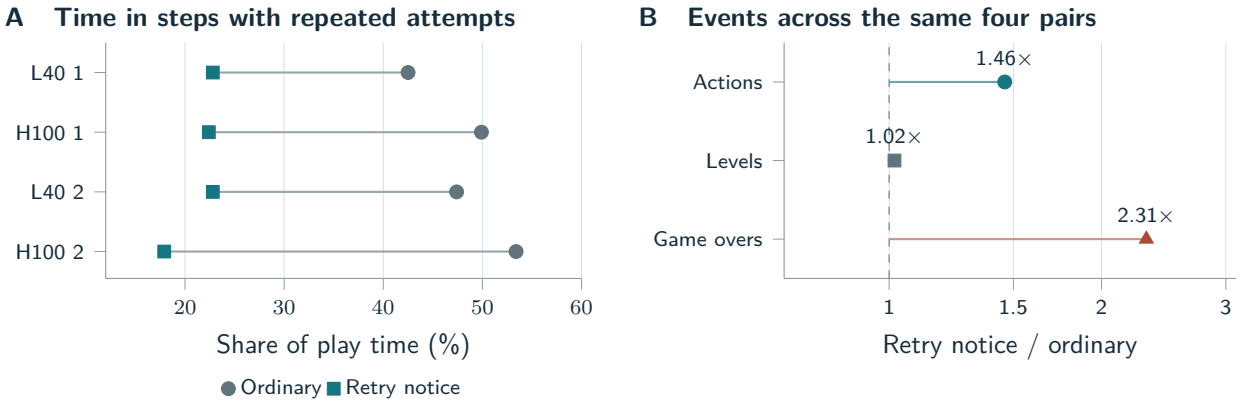


Figure 3. The notice reduced retries in every pair. Each line in A joins a hardware-matched pair. The points in B divide the pooled treatment count by its control count for each outcome. The underlying counts are 13,229 versus 9,071 actions, 116 versus 114 completed levels, and 125 versus 54 game-over events.

5.2 What happened during the recovered time?

Retry occupancy fell in every pair, from 42.5–53.4% of play time under the ordinary prompt to 17.9–22.8% with the notice (Figure 3A). The recovered time went into action: across the four pairs the agent executed 4,158 additional moves. It did not go into progress. The same pairs completed two additional levels and ran into 71 additional game overs. Per 1,000 actions, game overs rose from 5.95 to 9.45 while completions fell from 12.57 to 8.77. The time the agent had spent stalled was not all wasted; nudged to act, it walked into more traps than it cleared levels.

The game score did not improve either; it changed by −0.23 points per game on average (Appendix A).

6 What survives the next turn?

An agent may spend a turn discovering why an earlier move failed. Whether that discovery reaches the next turn is up to its software. In an earlier Duck system running Qwen3.6-27B [9], the default chat template quietly stripped the reasoning text from previous assistant turns. The next prompt kept the tool call and its result, but not the analysis that led to it. Enabling the template's `preserve_thinking` setting brought the reasoning back. (In the retry experiment above, earlier reasoning was already visible.)

We found this while tuning sampling temperature, which controls how variable the model's output can be. With earlier reasoning stripped, two runs at temperature 0.6 scored 0.23 and 0.34, and a run at temperature 1.0 scored 0.93. A noisier sampler looked like the cure for poor exploration. We repeated the comparison with the reasoning restored.

Restoring the reasoning tripled the score at temperature 0.6: eight runs averaged 0.92, against 0.285 without it. At temperature 1.0 the run with retention scored only 0.55, so the temperature ranking reversed (Figure 4).

The reversal suggests that randomness had been standing in for memory. An agent that has lost the rationale for a failed plan can be shaken out of it by a noisier sample; an agent that remembers why the plan failed is better served by focus. The practical lesson is to inspect the conversation as rendered for the model, not as saved in the log.

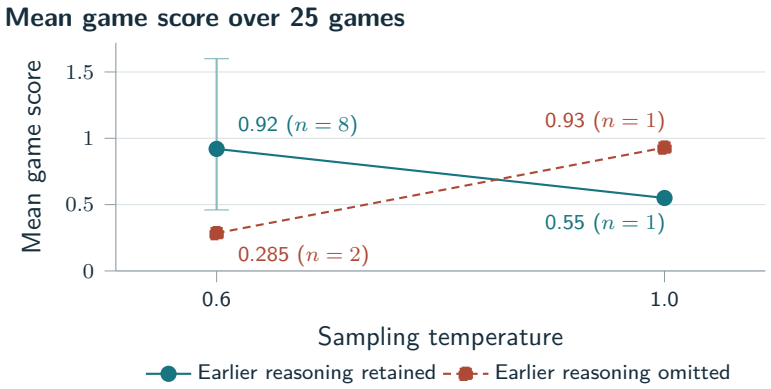


Figure 4. Restoring earlier reasoning tripled the score and reversed the temperature ranking. Each point summarizes complete runs over the 25 games at the indicated temperature. The vertical bar spans the eight observed scores at temperature 0.6 (0.46–1.60).

7 Long attempts are not stalled

A long stay on one level can mean that a solver is recycling an idea. It can also mean that it has ruled out several hypotheses and is closing in on the answer. We tested a restart built on the first reading, adapting a public implementation by heliosli [10]. Once per level, after a set number of generated tokens without a completion, it cleared the reasoning context and the saved Python functions and asked the agent to review the level afresh. The game continued from the existing board, and the interaction history, death record, and counters remained available. We compared total completions after ten hours with earlier runs without a restart, which averaged 52.7 levels on both L40 and H100 GPUs.

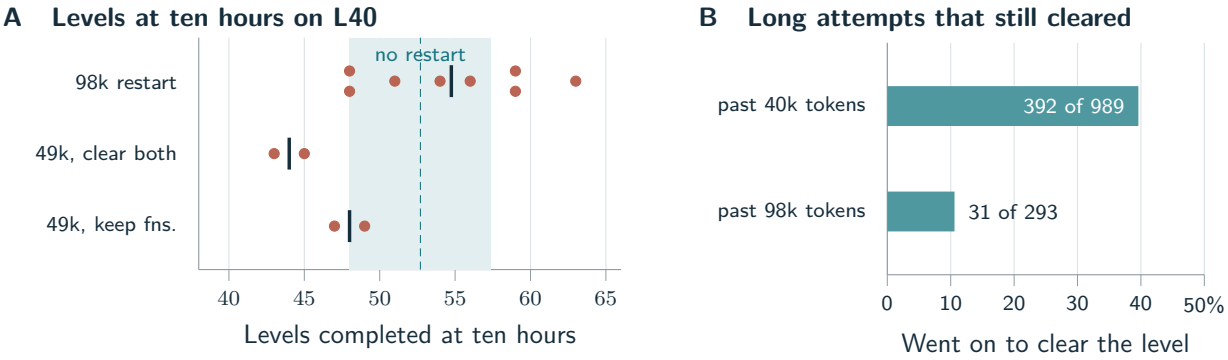


Figure 5. Restarting long attempts cost levels, and earlier restarts cost more. A shows every complete L40 run at ten hours (dots) and each group's mean (ticks) for a restart at 98,304 tokens and at 49,152 tokens, with the saved functions cleared or kept. The shaded band is the mean of runs without a restart plus or minus one standard deviation (52.7 ± 4.7). B shows how often level attempts by agents without a restart went on to clear the level after passing each token count.

The restart did not help at its original trigger, and firing it earlier cost levels (Figure 5A). At 98,304 tokens it made no measurable difference: eight L40 runs finished 2.05 levels above the reference, well inside the spread of runs without a restart. At 49,152 tokens the two L40 runs completed 45 and 43 levels, 8.7 below the reference, and took 24% and 30% fewer actions; a single H100 run ended 12.7 levels below its reference.

The agents without a restart show why. Long attempts are often not stalled. Among L40 level attempts that had passed 40,000 generated tokens, two in five (392 of 989) still went on to clear the level; past 98,000 tokens, about one in ten (31 of 293) did (Figure 5B). A restart at 49,152 tokens cuts into the stretch where many attempts are still on their way to a win.

Both the reasoning and the saved functions carried value. In a final variant we kept the saved Python functions when clearing reasoning at 49,152 tokens. The two L40 runs finished at 47 and 49 levels, and the two complete H100 runs at 48 and 61. Keeping the functions recovered about half of the L40 loss (48.0 levels, against 44.0 with both cleared and 52.7 without a restart), so losing the functions and losing the reasoning each cost roughly half.

8 Learning from runs that succeeded

A different way to reuse experience is to train on it. The public games give few explicit hints about their goals, but a completed level identifies a sequence that worked. We fine-tuned Qwen3.6-27B with a low-rank adapter [11], a small set of trainable weights, on 3,172 samples from successful trajectories in six games, and tested it on the nineteen other games. Segments leading to a second-level completion received four times their original weight.

A winning trajectory is a biased teacher. It can contain exploration, but only exploration that paid off, and it ends with the short, decisive moves of a player who has already found the rule. A solver starting a new game does not yet know which of its hypotheses will pay off.

Outcome	Base agent	Success-trained adapter
Success on nineteen held-out games		
Game-runs	502	37
First level completed	235/502 (46.8%)	4/37 (10.8%)
How the attempts were played		
Game-runs	394	37
Game actions per run, median	82	25
Generated tokens per action, median	827	588
Action budget used, median	16%	5%

Table 2. Attempts became shorter on games excluded from adapter training. A game-run is one play of one game, and all treated game-runs use the same first-epoch adapter.

The adapter played as if it had already found the rule. Its median attempt took 25 actions and used 5% of the action budget, against 82 actions and 16% for the base model (Table 2); one attempt ended after just ten actions. First-level success on the held-out games fell from 46.8% (235 of 502 base game-runs) to 10.8% (4 of 37). Only one of the nineteen per-game score comparisons favored the adapter, and the mean difference was -1.14 points.

The adapter learned the winner's pace without the winner's knowledge. It stopped testing hypotheses before it had one worth acting on.

9 Protect the thinking

Table 1 lines up the six changes. The ones that made the agent look more efficient (fewer tokens per turn, fewer repeated analyses, earlier restarts, shorter attempts) lost levels or gained almost none. The two that gave the model more of its own reasoning gained. The sharpest contrast is between Swift and the serving change. Both gave the agent more turns. Swift paid for them with shorter reasoning and finished 15 levels behind; the serving change paid for them with less waiting, kept the reasoning per action, and gained 32 levels over four pairs.

Our reading is that an ARC-AGI-3 level is solved by a chain of hypotheses: what a key does, which object matters, what counts as the goal. The first link is cheap, since the mechanics can be pinned down from a few hundred moves (Section 2); the reasoning goes into the later ones. Each link is tested by a probe or a move and then kept or discarded, and the record of why a hypothesis

failed is what stops the agent from testing it again. Cutting a turn short, wiping the context, or training the agent to play like someone who already knows the rule all break that chain partway through. Actions per hour rise because the agent stops paying for the expensive part. Swift's losses were largest in the games where the base model went deepest, which is where such chains are longest.

The chain is most fragile when the level changes. In the Duck system of Section 6, 74% of runs that cleared a game's first level still had more than half of their budget left, yet only 16% cleared the second; the agent carried the old level's action model and goal onto a board where they no longer held. We added a one-time instruction at each new level: compare the board with the previous one, retest what each action does with the cheapest probe, and only then propose a goal, with the full history kept. Runs with this instruction, alone or combined with a menu of goal templates and board-modeling helpers, played a game four or more levels deep five times; none of 1,714 game-runs without it ever did (Appendix A). Helping the chain at its weakest link let it run further, while every change that cut it short made it shorter.

The same pattern appears on a different benchmark, in a different kind of solver [12]. On ARC-AGI-2, a 27B reasoning model works on each puzzle alongside a grid solver. Answers it finished on its own, within the 64,000-token thinking limit used in competition, were right 37 times in 56 (66%); answers cut off at the limit and forced out were right 2 times in 32 (6%), so one long attempt beat two short ones. Shown the grid solver's candidate answers, the reasoner thought just as long but ended closer to them: on the puzzles where the grid solver was wrong, its accuracy fell from 31% to 16% and 24% in two runs. Cutting the reasoning short left it almost no value, and seeding it with another solver's answers took away a quarter to a half of it.

For anyone building an ARC-AGI-3 agent, the practical consequence is that the usual efficiency signals point the wrong way. Fewer tokens per action, more actions per hour, and less time on a single level all looked like progress in our runs, and none of them was. The reasoning behind each move is where levels are won.

10 Relation to prior work

Interaction and efficient reasoning. ReAct interleaves reasoning with actions and external observations [13]. Shen et al. scale the number of interaction steps rather than the reasoning within each step, and find the two complementary for web agents [14]. On ARC-AGI-3 the trade cut both ways: extra turns bought by thinking less per turn lost levels, while extra turns bought by waiting less gained them. Swift comes from a line of work on overthinking, which finds that reasoning models spend many tokens on easy problems and that much of this can be trimmed with little loss on static benchmarks [15, 16]. Work on test-time scaling points the other way: more computation per problem can substitute for a larger model [17], and forcing a model to keep reasoning when it tries to stop improves accuracy [18]. In long interactive play our results side with the second line; the savings came back as lost levels. On ARC-AGI-3, Bigio and Sanders report gains from retaining reasoning and compacting context [19], and our history experiment agrees on retention.

Representing experience across decisions. Rodionov studies coding agents that write executable Python models of games and verify them against observations [20]; the Duck agent instead carries its current hypothesis as a textual rule note. A program can be run to test a move, while text can only guide which move the model thinks to test. Reflexion keeps verbal reflections on failed attempts so that later attempts can avoid them [21]; the history experiment in Section 6 is a case in which a chat template was silently deleting that kind of record. Appendix B reports our own attempt at a learned next-frame predictor. Wu et al. traced the scripts and notes of another ARC-AGI-3 coding agent and found that the agent often reconstructed knowledge in new scripts and seldom called the old ones [22]. That distinction matters for the restart comparison.

A surviving game log can contain evidence needed to reconstruct a useful function even after the function itself has been erased.

Learning from trajectories. Training on a model's own successful reasoning traces is a standard way to improve reasoning [23], and Chen et al.'s ATLAS selects critical steps from expert trajectories and studies the generalization cost of cloning whole trajectories [24]. Our experiments document the contraction of attempts under one success-conditioned training recipe on games withheld from that recipe. Lehmann and Vahdati let language models revise ARC-AGI-3 harnesses and study the cost and generalization of the result [25]; our findings suggest that such revisions should leave the reasoning behind each move intact.

11 Future work

Our experiments ran on the 25 public games with 27B models. The natural next test is whether the pattern holds on the hidden games and on larger models such as the Qwen3.8-Flash-Next in our leaderboard entry. The level-change result also points to a direction: aids that help the agent re-ground when a level changes, rather than ones that cut its reasoning short.

12 Conclusion

Three hours into play, the model that thought less was winning. Ten hours in, it was 15 levels behind. Every other way we found to cut the thinking short, by nudging the agent past a stalled board, restarting long attempts, or teaching it to play like a winner, failed in the same direction, while giving the model more of its own reasoning paid off. For an agent that must work out unfamiliar rules, the expensive part of a turn is the useful part. Don't interrupt the thinker.

Acknowledgments

We thank the ARC Prize Foundation for making the ARC-AGI-3 benchmark and its public demonstration games available. This work used computational resources provided by the High Performance Computing Cluster at Brandeis University.

References

[1] ARC Prize Foundation. 2026. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. arXiv:2603.24621.

[2] François Chollet. 2019. On the Measure of Intelligence. arXiv:1911.01547.

[3] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. 2026. ARC Prize 2025: Technical Report. arXiv:2601.10904.

[4] UkisAI. 2026. Swift-1.5-Qwen3.8-27B. Model card.

[5] Qwen Team. 2026. Qwen3.8-27B. Model card.

[6] Jeroen Cottaar and Tufa Labs. 2026. The Duck. ARC-AGI-3 solver and inference harness.

[7] Daniel Franzen. 2026. ARC-AGI-3 Milestone 2 solution built on the Duck harness. Technical report and software.

[8] Woosuk Kwon et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP.

[9] Qwen Team. 2026. Qwen3.6-27B. Model card.

[10] heliosli. 2026. Try merging it—you might get lucky and win gold. Kaggle notebook, version 4, cell 5.

[11] Edward J. Hu et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. ICLR.

[12] Xinyi Fang. 2026. The Agreement Trap: When ARC-AGI-2 Solvers Agree on the Wrong Grid. Companion paper, ARC Prize 2026.

[13] Shunyu Yao et al. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR.

[14] Junhong Shen et al. 2025. Thinking vs. Doing: Agents that Reason by Scaling Test-Time Interaction. arXiv:2506.07976.

[15] Xingyu Chen et al. 2024. Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs. arXiv:2412.21187.

[16] Yang Sui et al. 2025. Stop Overthinking: A Survey on Efficient Reasoning for Large Language Models. arXiv:2503.16419.

[17] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv:2408.03314.

[18] Niklas Muennighoff et al. 2025. s1: Simple Test-Time Scaling. arXiv:2501.19393.

[19] Ilan Bigio and Ted Sanders. 2026. How enabling two settings tripled our scores on the ARC-AGI-3 benchmark. OpenAI.

[20] Sergey Rodionov. 2026. Executable World Models for ARC-AGI-3 in the Era of Coding Agents. arXiv:2605.05138.

[21] Noah Shinn et al. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS.

[22] Chen Wu, Josh Passenger, and Yin Song. 2026. Tracing the Thoughts of a Coding Agent Playing ARC-AGI-3: Lessons for Continual Learning. arXiv:2610.11450.

[23] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. 2022. STaR: Bootstrapping Reasoning With Reasoning. NeurIPS.

[24] Zhixun Chen et al. 2025. ATLAS: Agent Tuning via Learning Critical Steps. Findings of ACL.

[25] Jens Lehmann and Sahar Vahdati. 2026. Can LLMs Improve Their Own Harness? A Controlled Study on ARC-AGI-3. arXiv:2610.30499.

[26] Alex Nichol, Joshua Achiam, and John Schulman. 2018. On First-Order Meta-Learning Algorithms. arXiv:1803.02999.

A Experimental protocols and run-level observations

A.1 The submitted system

Our leaderboard entry is version 2 of the notebook squaree/franzen-m2-fork (submission 56836202, public score 33.09; a second run of the same version, submission 57785512, scored 32.97). It runs Franzen's solver with Qwen3.8-Flash-Next, quantized to 4-bit weights with AutoRound, and a separate quantized draft model for speculative decoding. An SGLang server holds the key–value cache in FP8. All 110 hidden games open at once, a priority gate lets ten requests generate at a time, and every game stops at a single deadline 532 minutes after the start. A decision step yields after 2,048 generated tokens. Each game's history grows by appending until it fills a 128K-token window, at which point older turns are trimmed to about 57K. Sampling temperature is 0.7.

The submitted system keeps the thinking in the ways this paper finds that matter. It passes earlier reasoning back to the model (Section 6), keeps saved Python functions across turns and does not restart long attempts (Section 7), and admits no more requests than the server runs at once, so prefix caching holds (Section 4). In a test run of the notebook on our account, with 25 minutes per game, 94.2% of prompt tokens came from the prefix cache and a request was waiting in 1.8% of decoding steps.

Our version changes how the notebook locates its inputs. Kaggle mounts attached datasets, models, and competition files under one of two directory layouts, and the layout can differ between the run that saves a notebook and the rerun that scores it. Our version resolves each of the five inputs (the wheel cache, the model weights, the draft model, the source bundle, and the competition files) under either layout. Before this change, the first scored run of the notebook on our account returned 0.00; with it, the first scored run returned 28.78.

A.2 Checkpoint replacement

The two checkpoints ran under vLLM 0.19 with FP8 weights, an FP8 key–value cache, and two L40 GPUs using tensor parallelism. We converted Swift with the same dynamic FP8 recipe used for the base. Sampling used temperature 0.7, top-p 0.95, and top-k 20. The solver maintained 25 game sessions and admitted ten inference streams. Play time excludes model loading. Median output per action-producing turn is computed within each run; the base value is the median of the 23 run medians.

A.3 Retry prompts and inference delivery

The repeat-prompt comparison used Qwen3.8-27B with FP8 weights, visual observations, vLLM, and no fixed sampling seed. The solver hashed the ordinary user prompt before annotation and excluded request errors from retry tracking. The notice was added only when the ordinary prompt matched the previous ordinary prompt, and an executed action cleared the retry counter. The saved history kept the ordinary prompt, so notices did not accumulate across turns. The summary of previously attempted probes had limits of six entries and 160 characters. Broad retry occupancy includes the entire duration of steps with at least three attempts. An independent byte-repeat diagnostic looked for long identical spans of reasoning.

The suite score is a sum of game scores over 25 games. We divide each treatment-minus-control sum by 25 and report the mean and standard error across the five pairs (Table 3). The event counts in Figure 3B come from the first four pairs.

The separate 132-minute serving comparison launched all 25 games together. The four pairs cleared 8 versus 16, 16 versus 13, 7 versus 18, and 10 versus 26 levels (old setup first in each pair). The mean paired increase was 8.0 ± 4.02 levels (standard error). In the turn-allowance test, one 110-second run delivered 1,240 tokens per turn and six completed levels, compared with 1,099 and 1,199 tokens per turn and eight and seven levels in the two historical 60-second runs.

The caching tests separated the two halves of the serving change. Eight games ran with eight

Matched pair	Ordinary prompt	Retry notice	Mean difference
L40, first	133.26	166.75	+1.3396
H100, first	162.40	129.19	−1.3284
L40, second	95.78	111.47	+0.6276
H100, second	82.68	88.57	+0.2356
L40, third	162.40	112.30	−2.0040

Table 3. Mean game-score contrasts for the retry prompt. The middle columns sum scores over 25 games. The final column gives the treatment-minus-control difference per game.

concurrent sequences, so no request waited; caching was the only difference between arms. Across nine pairs, the cached arm cleared −5, +4, +4, +3, +5, +2, +8, +4, and +5 levels relative to its partner (mean +3.33). We measure cache use as the share of prompt tokens served from the cache. In the queued test, all 25 games ran with eight concurrent sequences and a memory pool of about five 32K contexts, leaving 16–18 requests waiting. The cache supplied 2.9% of prompt tokens on an H100 and 12.8% on L40s over the run, nearly all of it in the first 15–25 minutes; after the contexts passed about 20,000 tokens, the share of new prompts served from the cache was 0–2%. Enlarging the pool to about seven 32K contexts left it at 1.0%.

A.4 History retention, restarts, and imitation

The temperature comparison used complete, 25-game runs on an H100. At temperature 0.6, the two runs without earlier reasoning scored 0.23 and 0.34, and the eight runs with it averaged 0.92 (range 0.46–1.60). Figure 4 therefore shows the full pool's mean and observed range.

The restart treatments used the Qwen3.8-27B Franzen agent. Each GPU type has its own reference runs without restarts. Table 4 lists the ten-hour levels of every run.

Condition	L40	H100
Unmodified reference, mean (SD)	52.7 (4.7), $n = 29$	52.7 (4.4), $n = 6$
98,304 tokens, clear reasoning and functions	54, 59, 63, 59, 48, 48, 51, 56	57, 52, 40
49,152 tokens, clear reasoning and functions	45, 43	40
49,152 tokens, retain functions	47, 49	48, 61

Table 4. All complete restart runs, in completed levels at ten hours.

For Figure 5B, token counts come from per-game progress logs sampled about every ten minutes, and a level counts as cleared when the next level begins.

The imitation adapter was trained on the BF16 Qwen3.6-27B base with rank 32, scaling 16, dropout 0.1, a learning rate of 2×10^{-5}, gradient accumulation of eight, and an 8,192-token sequence cutoff. The task evaluation used the first-epoch adapter. The adapter performed poorly both in BF16 and in the FP8 form used for play, so the drop is not a quantization effect. Per-game comparisons pair the adapter with the base model on each of the nineteen held-out games; first-level fractions count individual game-runs.

A.5 Level-change instruction

These runs used the Duck system of Section 6 with Qwen3.6-27B and earlier reasoning retained, on the 25 public games. The instruction was injected once, into the first analysis turn after the completed-level counter rose. It asked the agent to map the new board and compare it with the previous layout, to retest the action model with the cheapest discriminating probe, and only then to propose goals and plans; the full message history was kept. The combined configuration added a menu of goal templates for the agent to check against and a set of helper functions for modeling the

board. Runs with the instruction alone contributed 384 game-runs and the combined configuration 400. The five runs that cleared four or more levels in one game were three on `ft09`, one on `sb26`, and one on another game; the 1,714 control game-runs contained none (one-sided Fisher exact $p \approx 0.002$). On `ft09`, the combined configuration averaged 8.85 game-score points more than its controls, and across all games the paired difference was +0.38 points per game.

B When a predictor is useful for a new game

B.1 What a move does is determined by the board

We replayed the same 300 actions from a reset twice on `sp80` and compared frame hashes; all 300 frames matched. To ask whether the visible board determines what happens next, we treated the next color of each cell as a function of the action and of the square window centered on that cell. Among cells that changed, we counted how often the same window and action were followed by different next colors (Table 5). Each game contributed 500–600 random moves. Only windows that occur more than once can disagree, so the zeros are not an artifact of sparse data: at 41 cells across, `sp80` had 5,837 repeated windows and none disagreed, and at 25 cells across, `cd82` had 8,170 and none disagreed. The window that suffices varies by game, from 7 cells for `ar25` to 41 for `sp80`.

	Window width (cells)				
Game	7	17	25	33	41
`ar25`	0.0	0.0	–	0.0	–
`wa30`	4.5	0.0	–	0.0	–
`cd82`	33.0	2.3	0.0	–	0.0
`sp80`	36.0	20.1	5.6	–	0.0
`lf52`	35.9	25.2	–	0.0	–
`r11l`	41.3	18.3	–	3.0	–

Table 5. Share of changed cells whose next color was ambiguous (%). An entry is ambiguous when the same window and action were also followed by a different color. A dash marks a width we did not measure.

B.2 Predicting a changed cell and predicting a usable board

Before turning to language-model agents, we tried to learn what actions do by training a *next-frame predictor*, a learned world model. Each training example contains a board, an action, and the board that follows. A residual convolutional network with about 1.6 million parameters receives the colors, the action identity, and spatial channels for a click location, and predicts the colors of the next board. Repeatedly feeding its predictions back into it lets a player preview a sequence of moves.

The residual next-frame predictor used group normalization and a receptive field of 41 cells across. The frame was encoded as per-color channels. For a click, spatial channels specified where on the board the action was directed. We evaluated predictions in several ways. Changed-cell recall measures the share of actually changed cells that are detected. Precision asks how many of the proposed changes really occurred. Exact-frame accuracy requires every cell in the predicted frame to be correct.

One early predictor detected 97.9% of the cells that changed, yet only 44.5% of its predicted changes were real. It introduced on average 91.7 false positive pixels per frame. A separate loss-weight scan on game `cd82` illustrates the tradeoff. Increasing the changed-pixel weight from one to ten raised recall from 0.8869 to 0.968, while exact-frame accuracy fell from 0.343 to 0.098. This matters for lookahead because a spurious object can become the input to the next simulated move.

In a separate rollout evaluation, the unweighted predictor on `sp80` gave exact-frame fractions of 0.767, 0.517, 0.250, and 0 at depths one, two, three, and five. On `ar25`, the fraction stayed near 0.20 through depth ten. The latter game already had a low one-step value. The two examples distinguish a predictor that loses accuracy over a few imagined moves from one that starts inaccurate.

We also tested a player that used a prediction immediately. It spent its first 120 actions exploring with a heuristic that clicks at random on objects rather than anywhere on the board, fitted a small predictor to those transitions, then scored each candidate move by the amount of screen change it predicted. The player usually took the highest-ranked move and sometimes explored another one. Over 25 games and 20 seeds, it averaged 0.090 completed levels per game-run, against 0.100 for the heuristic alone. A move that changes many pixels is valuable only if those changes help reveal or achieve the goal.

B.3 What transfers before and after adaptation?

To test transfer, we held out one of the 25 games at a time and pretrained on transitions from the other 24. Each held-out game supplied 300 local transitions for adaptation. With 1,500 pretraining transitions per source game, mean exact-frame accuracy on the held-out game was 0.0057 before adaptation and 0.4526 afterward. Training from scratch with the same local data gave 0.3531. With 6,000 transitions per source game the corresponding values were 0.0086, 0.4608, and 0.3530. Pretraining supplied a useful starting point once local evidence was available, even though the predictor performed poorly on a new game at first contact.

Reptile changes the pretraining objective by training inside a sampled game, then moving the common initialization partway toward the adapted weights [26]. To compare it with ordinary pretraining we matched the architecture, gradient-step count, data cache, and local adaptation budget. With the first training seed, ordinary pretraining reached 0.4365 and Reptile 0.4704 after adaptation. The mean of the 25 paired game contrasts was $+0.0339 \pm 0.0134$ (standard error across games).

At interpolation steps 0.05, 0.2, 0.5, and 1.0, the first seed's paired contrasts were +0.0287, +0.0339, +0.0024, and −0.0868. A step of 1.0 uses the latest task-adapted weights as the initialization for the next task. The parameter sweep and a separate inner-step ablation shared the first training realization. The five seed-level mean contrasts at the main setting were +0.0339, −0.0316, +0.0230, +0.0224, and +0.0148. Their mean was +0.0125 with standard deviation 0.0256.